COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010/Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	Jul 24, 2026	**DATE SUBMITTED:**	Aug 14, 2026
NAME:	Antonio, Christine Mae P.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HOA NO. 3.1 **LINKED LISTS**

1. PROCEDURE OUTPUT

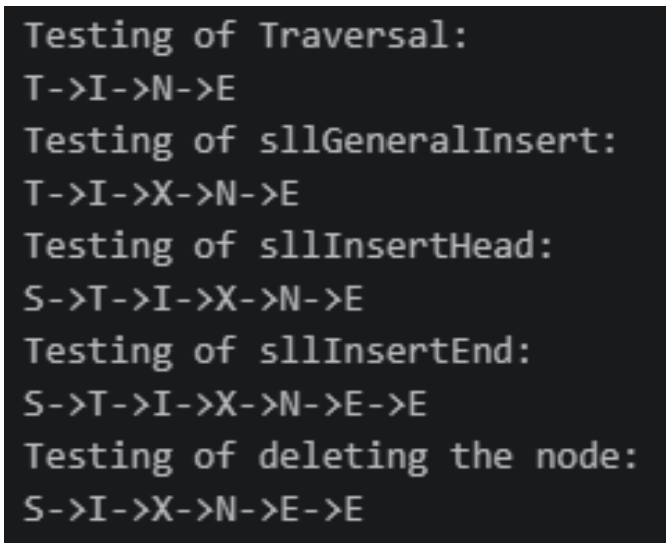
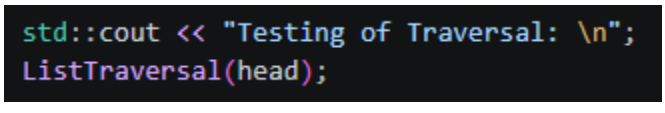
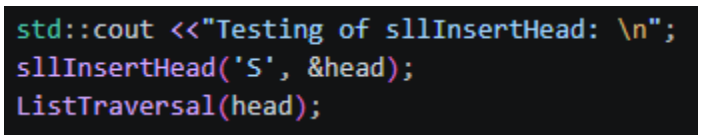
Screenshot	
Discussion	

Table 3-1. Output of Initial/Simple Implementation

Operation	**Screenshot**
Traversal	
Insertion at head	

Insertion at any part of the list	`std::cout << "Testing of sllGeneralInsert: \n";` `sllGeneralInsert('X', second);` `ListTraversal(head);`
Insertion at the end	`std::cout <<"Testing of sllInsertEnd: \n";` `sllInsertEnd('E', &head);` `ListTraversal(head);`
Deletion of a node	`std::cout << "Testing of deleting the node: \n";` `sllDeleteNode('T', &head);` `ListTraversal(head);`

Table 3-2. Code for the List Operations

a.

Source Code

```
11  //traverses through the single linked list
12  template <typename T>
13  void ListTraversal(SingleList<T>* head) {
14
15      //checks if the list is empty
16      if(head == nullptr) {
17          std::cout << "The list is empty.";
18          return;
19      }
20
21      //prints the list by looping until the head is nullptr
22      while(head != nullptr){
23          //prints the value of the head
24          std::cout << head->data;
25          //prints "->" if the next node is not nullptr
26          if(head->next != nullptr){
27              std::cout << "->";
28          }
29
30          //assigns the next node of head as the head
31          //this lets the head travel to the end of the list
32          head = head->next;
33      }
34      std::cout << std::endl;
35  }
```

Console

```
Testing of Traversal:
T->I->N->E
```

b.

Source Code

```
58  //insert head
59  template <typename T>
60  void sllInsertHead(T newData, SingleList<T>** currentHead){
61      SingleList<T>* newNode = new SingleList<T>;
62
63      newNode->data = newData;
64
65      //copies the currentHead to the next of NewNode
66      newNode->next = *currentHead;
67
68      //sets the currentHead to newNode
69      *currentHead = newNode;
70
71  }
```

Console

```
Testing of sllInsertHead:
S->T->I->X->N->E
```

c.

Source Code

```
37  //General Insert
38  //takes the value of the node and the previous node,
39  //where the new node will be inserted after it
40  template <typename T>
41  void sllGeneralInsert(T newData, SingleList<T>* prevNode){
42
43      //checks if the previous node is pointing to nullptr
44      if(prevNode == nullptr) {
45          std::cout << "Previous value cannot be null\n";
46          return;
47      }
48
49      SingleList<T>* newNode = new SingleList<T>;
50
51      newNode->data = newData; //assigns the input as the newNode value
52      newNode->next = prevNode->next; //copies the value next node of the new node as itself
53      prevNode->next = newNode; //replaces the current node as the new node
54
55  }
```

Console

```
Testing of sllGeneralInsert:
T->I->X->N->E
```

d.

Source Code

```
73  //inserts node at the end of the list
74  template <typename T>
75  void sllInsertEnd(T newData, SingleList<T>** currentHead){
76      SingleList<T>* newNode = new SingleList<T>;
77
78      newNode->data = newData;
79      newNode->next = nullptr;
80
81      //if the list is empty, the inserted node will be the head
82      if (*currentHead == nullptr) {
83          *currentHead = newNode;
84      }
85
86      //temporary storage that copes the current head
87      SingleList<T>* temp = *currentHead;
88      //checks if the current head is at the end of the list,
89      while (temp->next != nullptr) {
90          //if not, the next node will be set as the node
91          temp = temp->next;
92      }
93
94      //if the node is at the end of the list,
95      //adds the new node at the end of the list
96      temp->next = newNode;
97
98  }
```

Console

```
Testing of sllInsertEnd:
S->T->I->X->N->E->E
```

e.

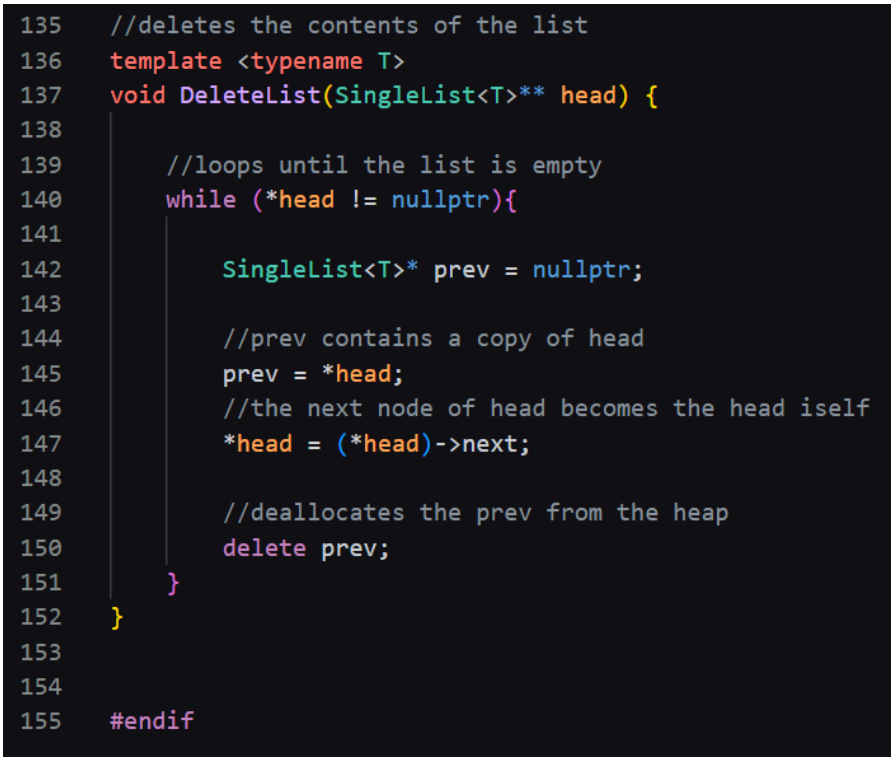
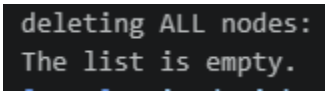
Source Code

```
100  //deletes a node from the list
101  template <typename T>
102  void sllDeleteNode(T findData, SingleList<T>** head){
103
104      //checks if the list is empty
105      if (*head == nullptr) return;
106
107      //sets the current node as the head
108      SingleList<T>* currNode = *head;
109      //makes the previous node point to nullptr
110      SingleList<T>* prevNode = nullptr;
111
112      //searches the list by checking
113      //if the node is not pointing to nullptr and
114      //if the node matches the value
115      while(currNode != nullptr && currNode->data != findData) {
116          //while false,
117          //sets the current node as the previous node
118          prevNode = currNode;
119          //moves the current node to its next node
120          currNode = currNode->next;
121
122      }
```

```
123
124          if (prevNode == nullptr) {
125              //moves the head to the front
126              *head = currNode->next;
127          } else {
128
129              prevNode->next = currNode->next;
130          }
131          delete currNode;
132      }
```

Console

```
Testing of deleting the node:
S->I->X->N->E->E
```

f.	**Source Code**	
	Console	

```
135   //deletes the contents of the list
136   template <typename T>
137   void DeleteList(SingleList<T>** head) {
138
139       //loops until the list is empty
140       while (*head != nullptr){
141
142           SingleList<T>* prev = nullptr;
143
144           //prev contains a copy of head
145           prev = *head;
146           //the next node of head becomes the head iself
147           *head = (*head)->next;
148
149           //deallocates the prev from the heap
150           delete prev;
151       }
152   }
153
154
155   #endif
```

```
deleting ALL nodes:
The list is empty.
```

Table 3-3. Code and Analysis for Singly Linked Lists

a.

Source Code

```
template <typename T>
void dllTraverse(DoubleList<T>* currentNode) {

    DoubleList<T> *tail;

    //check if list is empty
    if (currentNode == nullptr) {
        std::cout << "The list is empty." << std::endl;
        return;
    }

    //move forward
    std::cout << "Forward: \n";
    while (currentNode != nullptr) {
        std::cout << currentNode->data << " ";
        tail = currentNode;
        currentNode = currentNode->next;
    }

    //add next line
    std::cout << std::endl;

    //move backward
    std::cout << "Backward: \n";
    while (tail != nullptr) {
        std::cout << tail->data << " ";
        tail = tail->prev;
    }

    std::cout << std::endl;

}
```

Console

```
Testing the DLL Traversal:
Forward:
C P E
Backward:
E P C
```

b.

Source Code

```
template <typename T>
void dllInsertHead (T newData, DoubleList<T>** currentHead) {
    //creating a new node
    DoubleList<T>* newNode = CreateNewNode(newData);

    //new node should point to the current head
    newNode->next = *currentHead;

    //current Head should point back to the newNode
    (*currentHead)->prev = newNode;

    //update the pointer head
    *currentHead = newNode;
}
```

Console

```
Testing the insertion at the head node:
Forward:
X C P E
Backward:
E P C X
```

c.

Source Code

```
template <typename T>
void dllGeneralInsert(T newData, DoubleList<T>* previousNode){

    DoubleList<T>* newNode = CreateNewNode(newData);

    if (previousNode == nullptr) {
        std::cout << "Previous value cannot be null" << std::endl;
        return;
    }

    //store data in new node
    newNode->data = newData;

    newNode->next = previousNode->next;
    newNode ->prev = previousNode;

    //previous node to new node
    previousNode->next = newNode;

}
```

	Console	`Testing the DLL General Insertion:` `Forward:` `X C P G E O` `Backward:` `O E P C X`
d.	**Source Code**	`template <typename T>` `void dllInsertEnd(T newData, DoubleList<T>* currentHead) {` `  //create new node` `  DoubleList<T>* newNode = CreateNewNode(newData);` `  //traverse until last node reached` `  while (currentHead->next != nullptr) {` `  currentHead = currentHead->next;` `  }` `  //connect the last node to the new node` `  currentHead->next = newNode;` `  //connect the new node back to the last node` `  newNode->prev = currentHead;` `}`
	Console	`Testing the insertion at the end node:` `Forward:` `X C P E O` `Backward:` `O E P C X`

e.	**Source Code**	``` template <typename T> void dllDeleteNode(T findData, DoubleList<T>** head){ if (*head == nullptr) return; DoubleList<T>* currNode = *head; while(currNode != nullptr && currNode->data != findData) { currNode = currNode->next; } if (currNode == nullptr) return; if (currNode->prev == nullptr){ *head = currNode->next; if(*head != nullptr){ (*head)->prev = nullptr; } } else { currNode->prev->next = currNode->next; if(currNode->next != nullptr) { currNode->next->prev = currNode->prev; } } delete currNode; } ```
	Console	``` Testing the DLL Delete Node: Forward: X C P E O Backward: O E P C X ```

Table 3-3. Code and Analysis for Doubly Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem Title: Implementing a Song Playlist using Linked List
Source: Packt Publishing

Problem Description:

In this activity, we'll look at some applications for which a singly linked list is not enough or not convenient.

We will build a tweaked version that fits the application. We often encounter cases where we have to customize default implementations, such as when looping songs in a music player or in games where multiple players take a turn one by one in a circle.

These applications have one common property – we traverse the elements of the sequence in a circular fashion. Thus, the node after the last node will be the first node while traversing the list. This is called a circular linked list.

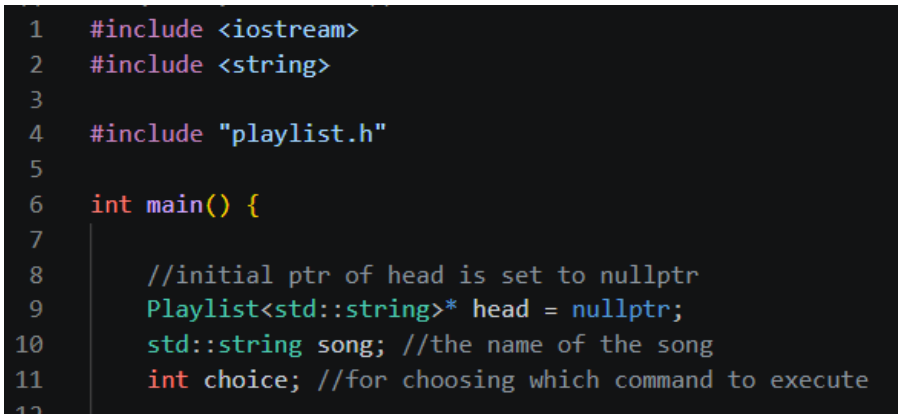
We'll take the use case of a music player. It should have following functions supported:

- Create a playlist using multiple songs.
- Add songs to the playlist.
- Remove a song from the playlist.
- Play songs in a loop (for this activity, we will print all the songs once).

Here are the steps to solve the problem:

- Design the basic structure that supports circular data representation.
- After that, implement the insert and delete functions in the structure.
- Implement a function for traversing the playlist.

The driver function should allow for common operations on a playlist such as: next, previous, play all songs, insert and remove.

Main	
Screenshots	

```cpp
#include <iostream>
#include <string>

#include "playlist.h"

int main() {

    //initial ptr of head is set to nullptr
    Playlist<std::string>* head = nullptr;
    std::string song; //the name of the song
    int choice; //for choosing which command to execute
```

```cpp
//loops until the input is 'x'
do {
    //the 'control panel'
    std::cout << "================================================================================" << std::endl;
    std::cout << "Select Command:\n(1)Add Song\n(2)Remove Song\n(3)Next\n(4)Previous\n(5)Display All Songs" << std::endl;
    std::cout << "================================================================================" << std::endl;
    std::cin >> choice;
```

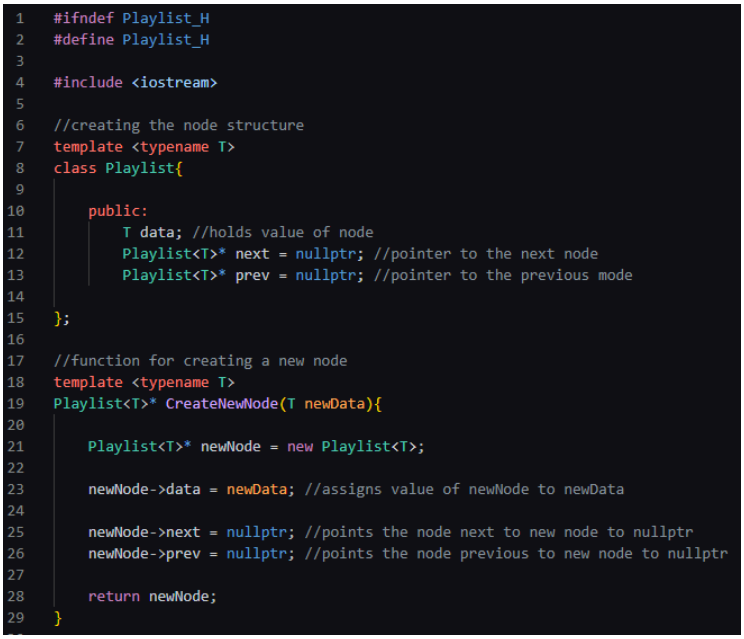
```
    //loops until the input is 'x'
    do {
        //the 'control panel'
        std::cout << "==========================" << std::endl;
        std::cout << "Select Command:\n(1)Add Song\n(2)Remove Song\n(3)Next\n(4)Previous\n(5)Display All Songs" << std::endl;
        std::cout << "==========================" << std::endl;
        std::cin >> choice;

        switch(choice){
            //adds a song to the playlist
            case 1:
                std::cout << "Add song: " << std::endl;
                std::cin >> song;

                AddSong<std::string>(song, &head);
            break;

            //removes the chosen song from the playlist
            case 2:
                std::cout << "Remove song: " << std::endl;
                std::cin >> song;
                RemoveSong<std::string>(song, &head);
            break;

            //moves the current song forward and becomes the tail
            //movese the previous song forward and becomes the head
            case 3:
                Next(&head);
            break;

            //moves the previous song to the front and becomes the head
            case 4:
                Prev(&head);
            break;

            case 5:
            //prints all the songs in order
                PlaylistTraverse(head);
            break;
        }
```

```
        } while (choice != 'x');
    }
```

Output

```
==========================
Select Command:
(1)Add Song
(2)Remove Song
(3)Next
(4)Previous
(5)Display All Songs
==========================
```

playlist.h

a.	**Screenshot**	

```cpp
#ifndef Playlist_H
#define Playlist_H

#include <iostream>

//creating the node structure
template <typename T>
class Playlist{

    public:
        T data; //holds value of node
        Playlist<T>* next = nullptr; //pointer to the next node
        Playlist<T>* prev = nullptr; //pointer to the previous mode

};

//function for creating a new node
template <typename T>
Playlist<T>* CreateNewNode(T newData){

    Playlist<T>* newNode = new Playlist<T>;

    newNode->data = newData; //assigns value of newNode to newData

    newNode->next = nullptr; //points the node next to new node to nullptr
    newNode->prev = nullptr; //points the node previous to new node to nullptr

    return newNode;
}
```

b.	**Screenshot**	

```cpp
//function to add songs
//accepts the song name and the head
template <typename T>
void AddSong(T newData, Playlist<T>** currentHead){

    //creates an object of CreateNewNode
    Playlist<T>* newNode = CreateNewNode(newData);

    //prints the value of new node
    std::cout << "Added " << newNode->data << " to playlist.\n";

    //checks if the playlist is empty
    if(*currentHead == nullptr){

        //sets the next and previous nodes of newNode to itself
        //this lets the song circle back to the playlist
        newNode->next = newNode;
        newNode->prev = newNode;

        //initially sets the head to the new node
        *currentHead = newNode;

        //prints the current song playing
        std::cout << "\nNow Playing: " << (*currentHead)->data << std::endl;

        return;
    }

    // Sets the tail to the head's previous pointer.
    Playlist<T>* tail = (*currentHead)->prev;

    //sets the node next to the new node as current head
    newNode->next = *currentHead;
    //sets the node previous to the new node as the previous node to head
    newNode->prev = tail;

    //updates the next node of the tail as the new node
    tail->next = newNode;

    //updates the previous node of the head as the new node
    (*currentHead)->prev = newNode;

    std::cout << "\nNow Playing: " << (*currentHead)->data << std::endl;
}
```

	Output	``` ========== Select Command: (1)Add Song (2)Remove Song (3)Next (4)Previous (5)Display All Songs ========== 1 Add song: Toy Added Toy to playlist. Now Playing: Toy ========== Select Command: (1)Add Song (2)Remove Song (3)Next (4)Previous (5)Display All Songs ========== 1 Add song: Vanillove! Added Vanillove! to playlist. Now Playing: Toy ```

c.

Screenshot

```cpp
//Traverses throughout and prints the list
template <typename T>
void PlaylistTraverse(Playlist<T>* currentHead) {

    //creates a variable of T and assign it as currentHead;
    Playlist<T>* currentNode = currentHead;

    //checks if the current node has nullptr
    //to determine if the playlist is empty
    if(currentNode == nullptr) {
        std::cout <<"Playlist is empty.";
        return;
    }

    //initializing index
    int i = 0;

    std::cout << "Playlist:\n";

    //prints the songs in the playlist
    do{
        i++;
        std::cout << i << ". " << currentNode->data << std::endl;
        //moves the current node to its next node
        currentNode = currentNode->next;

      //runs until the current node is equal to the current head,
      //which is equivalent to the 'tail' of the list because it is circular.
    } while(currentNode != currentHead);

    //prints the current song playing
    std::cout << "\nNow Playing: " << currentHead->data << std::endl;

    std::cout << std::endl;
}
```

Output

```
Select Command:
(1)Add Song
(2)Remove Song
(3)Next
(4)Previous
(5)Display All Songs
============================
5
Playlist:
1. Toy
2. Vanillove!

Now Playing: Toy
```

d.

Screenshot

```
113  //function for removing song off the playlist
114  //accepts the song name and head
115  template <typename T>
116  void RemoveSong(T findData, Playlist<T>** currentHead) {
117
118      //checks if the playlist is empty
119      if(*currentHead == nullptr){
120          std::cout << "Playlist is empty.";
121          return;
122      }
123      //creates a variable of T with the value of the head
124      Playlist<T>* currentNode = *currentHead;
125
126      //searches the list the song to be removed
127      do{
128          //if the node doesn't match the song, it moves to the next node
129          currentNode = currentNode->next;
130      } while (currentNode->data != findData && currentNode != *currentHead);
131
132      //checks whether the song exists in the playlist
133      if(currentNode->data != findData) {
134          std::cout << "Invalid input.";
135          return;
136      }
137      //checks if the playlist only has 1 song
138      if (currentNode->next == currentNode) {
139          *currentHead = nullptr;
140
141      // if there is more than 1 song in the playlist
142      } else {
143          //Link the previous node directly to the next node
144          currentNode->prev->next = currentNode->next;
145          //Link the next node directly back to the previous node,
146          currentNode->next->prev = currentNode->prev;
147
148          //checks if the song is the head
149          if (currentNode == *currentHead) {
150              //moves the head to the node next to it
151              *currentHead = currentNode->next;
152          }
153      }
154      std::cout << "Removing " << currentNode->data << " from playlist." << std::endl;
155      std::cout << "\nNow Playing: " << (*currentHead)->data << std::endl;
156      //deallocates the heap memory for currentNode
157      delete currentNode;
158  }
```

Output

```
============================
Select Command:
(1)Add Song
(2)Remove Song
(3)Next
(4)Previous
(5)Display All Songs
============================
2
Remove song:
Toy
Removing Toy from playlist.

Now Playing: Vanillove!
```

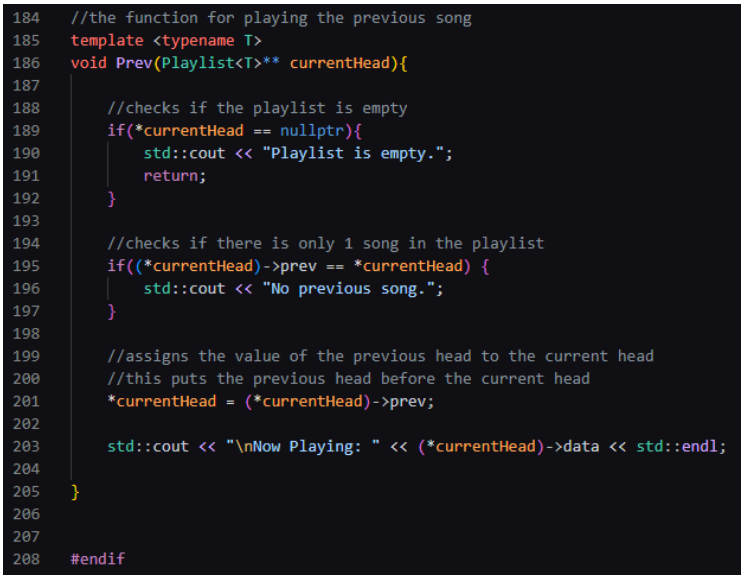
e.

Screenshot

```cpp
//the function for playing the next song in the playlist
template <typename T>
void Next(Playlist<T>** currentHead){

    //checks if the playlist is empty
    if(*currentHead == nullptr){
        std::cout << "Playlist is empty.";
        return;
    }

    //checks if the playlist contains only 1 song
    if((*currentHead)->next == *currentHead) {
        std::cout << "No song ahead.";
    }

    //the next node of the head becomes the head
    //the previous head becomes the tail
    *currentHead = (*currentHead)->next;

    std::cout << "\nNow Playing: " << (*currentHead)->data << std::endl;

}
```

Output

```
Playlist:
1. Vanillove!

Now Playing: Vanillove!

==========================
Select Command:
(1)Add Song
(2)Remove Song
(3)Next
(4)Previous
(5)Display All Songs
==========================
3
No song ahead.
Now Playing: Vanillove!
```

		
f.	**Screenshot**	

	Output	``` ========== Select Command: (1)Add Song (2)Remove Song (3)Next (4)Previous (5)Display All Songs ========== 4 Now Playing: Toy ========== Select Command: (1)Add Song (2)Remove Song (3)Next (4)Previous (5)Display All Songs ========== 5 Playlist: 1. Toy 2. No. 3. Trash ``` ``` Playlist: 1. Trash Now Playing: Trash ========== Select Command: (1)Add Song (2)Remove Song (3)Next (4)Previous (5)Display All Songs ========== 4 No previous song. Now Playing: Trash ```

Provide the following:
Summary of lessons learned
Analysis of the procedure
Analysis of the supplementary activity

Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?

Summary of lessons learned	The linked lists consist of nodes. Specifically, the single linked list nodes only points forward; The double linked list nodes can point either direction–backwards and forwards. In the circular linked list, the head points to its tail, and the tail points to its head.
Analysis of the procedure	The procedures cover single linked list and double linked list. Both of those used STL. They also have multiple operations: ListTraversal() - traverses through the list by going through each node and displaying their values GeneralInsert() - Allows the insertion of nodes wherever in the list by going through the node's position until it reaches the intended position. InsertHead() - inserts the node only as the head of the list. InsertEnd() - inserts the node only at the end of the list by going through the list until the head points to nullptr. DeleteNode() - deletes the node by deallocating it from the heap. DeleteList() - similar to ListTraversal, except it deletes the node after it has been gone through.
Analysis of the supplementary activity	The playlist used the circular linked list. I had created PlaylistTraversal(), AddSong(), RemoveSong(), Next(), and Prev(). PlaylistTraversal() - traverses through the playlist and prints it. AddSong() - the added song will always be at the end of the list. RemoveSong() - this function lets removal of songs at any position by searching through the list and checking the node value if they match the input value. Prev() - the current node becomes the previous node.
Concluding statement/feedback	It took me a long time to understand how the linked lists are structures, but learning them did improve my logical thinking. I had the hardest time implementing the RemoveSong() function in the supplementary activity because it is circular.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Claude.AI

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Idea generation, code assistance

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of

concepts, etc.

Extent of Use: Provide an example of what the outputs should look like, error detection

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.